

摘要：

DeepMind的Gemini预训练负责人 Vlad Feinberg自曝研发内幕：5人团队为抢跑 DeepSeek，在硅谷和巴黎双城倒班、40天几乎不眠不休死磕训练。他不认为人工智能会取代所有人的工作，原因在于，人类在组织里承担的一个关键角色，是构成一张信任网络。

Google DeepMind 的 Gemini 预训练主管 Vlad Feinberg，最近在一档播客里聊了聊他的日常。

在大众的想象中，顶尖实验室的研究员每天都在推导颠覆性的算法。但 Vlad 说，他职业生涯最重要的一笔奖金，是谷歌传奇人物 Jeff Dean 亲手发给他的——当时他刚入职 Google Brain，没有像当年同样在谷歌的 Transformer 作者们一样，去写那些能发到顶级会议上的第一作者论文，而是默默干了几天最脏的活：**调整编译器和超参数，解决显存溢出，把一个叫 SFT 的微调任务塞进了一堆老旧的 TPU 卡里，这才让第一代 Bard 勉强跑通。**



这种“干脏活”的工程体验，才是这轮大模型竞争最真实的样子。**Gemini 2.0 出来的时候，外界都在赞叹它作为一个 MoE 模型有多神奇。但 Vlad 透露，背后其实只有 5 个人在顶着。**

算力卡随时会挂，数据索引随时会断，为了不白白浪费几百万美元的算力费，他们只能在硅谷和巴黎两个大区之间 24 小时倒班，不眠不休地死磕了 40 天。甚至在 DeepSeek-V3 爆红、华尔街日报制作表格拉踩谷歌已经落后时，Vlad 也是哭笑不得——**媒体为了制造爆款新闻，在表格里故意删掉了（elided）排名其实高居第一的 Gemini 2.0 Flash Thinking。**

对于甚嚣尘上的“程序员要失业”的恐慌，这位主管给出了一个很干脆的观点：AI 永远无法被“吊销律师执照”，因为它不具备主体资格，无法承担法律责任，所以人类永远要为它的产出签字并背书。

他的组里有一个叫 Nate Lintz 的普通工程师，之前在搜索部门写后端基础架构，就是靠着在业务里帮大模型落地，解决最具体的推理开销，**最终内部转岗到 DeepMind 成了技术支柱。**



如果你也想去，Vlad 在他的博客里放了一个“硬核作业”（手写一个 Transformer 并手算 Scaling Laws 录成视频发给他），做完了他直接面你。以下是这次谈话里，他聊到的几个极其真实的行业细节：

- 。法律大模型可以背下所有判例，但它不能代表你出庭，因为它无法被“吊销执照”。职业的底层逻辑是责任和信任的分配。因为 AI 无法承担法律后果，代码的终点永远需要一个具体的人来签字、背书并承担责任。这才是程序员不会被替代的终极底线。
- 。写再牛逼的学术论文，都不如帮团队省下几张卡的显存。很多眼高手低的程序员在 AI 时代迷失在理论和框架中。但在研发一线，最容易拿奖金的能力，是那些不体面的“重体力活”——优化编译器、调试超参、在有限的芯片里榨出最后一丝算力。这种扎实的工程能力，才是跨越周期的硬通货。
- 。写搜索基础架构的普通码农，也是能一步步逆袭进 DeepMind 的。团队核心成员 Nate Lintz 曾只是一个写后端搜索的普通工程师，他没有高大上的 AI 背景，但通过在组内帮产品落地 LLM，默默解决大模型在搜索业务里最头疼的推理和算力开销，在实战中摸清了底层架构，最终顺理成章地转岗进入 DeepMind 并主导了核心推理设计。
- 。华尔街日报为了拉踩谷歌，在对比 DeepSeek 时故意在表格里删掉了排名第一的 Gemini 模型。当媒体为了制造“中国开源超越美国大厂”的新闻而兴奋时，他们有意隐去了当时在 LMSYS 榜单上高居榜首的 Gemini 2.0 Flash Thinking。真实的技术对决背后没有神话，只有 5 个工程师在硅谷和巴黎 24 小时倒班、死磕 40 天的硬撑。

你没法把研究和落地分成两种人

主持人：你写过一篇题为《如何进入前沿实验室工作》的文章。现在前沿实验室最需要的，到底是什么样的能力？也许我们可以先聊聊，这类工作的整体轮廓是什么。

Vlad：现在前沿实验室需要的能力，其实覆盖范围非常广。大语言模型这种东西，和研究、和产品之间的关系，比过去的机器学习要紧密得多，所以它会牵动很多完全不同的方向。我写那篇文章，并不是想开一份面面俱到的清单，而是想提出几个比较具体、可操作的方向，说明实验室会在哪些能力上有很强需求。

我重点写的是内核开发和底层工程，也就是怎样在真实运行环境中提升大语言模型的执行效率。我看到这类能力，在几乎所有前沿实验室，以及实验室内部很多项目里，需求都非常强。所以这是一个特别值得单独点出来的方向。更具体一点说，每当我们做研究项目，需要修改神

经网络架构，或者重新思考服务方式，比如怎样把键值缓存做得更好，类似这样的事情，整个技术栈上的人都必须有能力把这些新方法高效地实现出来。

而所有这些变化的核心循环，本质上都是在制造能够在大规模场景下运行的软件系统：它要有高吞吐、低延迟。这其实是一类非常基础的工作，和传统后端工程的思维是紧密相连的。所以我觉得，这对很多人来说都是一个非常开放、非常值得专攻的方向。

主持人：我有些朋友在 OpenAI 和 Anthropic 工作，他们跟我说，那边会区分偏应用的组织 and 偏研究的组织。我想知道 Google DeepMind 里是不是也有类似区分？如果有的话，你能不能讲讲这两者的差别？

Vlad：我们内部确实有不同的重点方向。比如在 Google DeepMind，就有团队专门研究怎样用 Gemini 这样的语言模型，更好地改进搜索结果。从某种意义上讲，这可以算是语言模型的一种应用化方向。

但我其实不太愿意把这种区分画得特别死，因为**把模型整合进真实产品，本身就需要很多非常硬核的研究工作。**就拿我刚才说的那个例子来说，为了让模型真正服务搜索，你得投入大量精力，确保模型回答的是事实，能引用来源，能给出非常精确、非常有依据的答案。

你还得评估这些来源本身的质量，确保它不是讽刺、不是玩笑、不是不可靠的信息。我觉得这恰恰说明，即便是在非常面向产品、非常“应用化”的人工智能方向里，你其实仍然是在做研究。

当然，另一方面，也确实存在那种更经典意义上的语言模型研究团队，比如做预训练、做后训练。这些在 Google DeepMind 内部依然是比较独立的团队，目标就是打造业内最先进的模型。

也就是更纯粹意义上的研究。不过我还是要补一句：我们做的“纯研究”之所以有意义，前提是它最后能被真正实现出来。所以我们既要负责把模型交付出来，确保训练稳定推进，某种程度上像训练任务的运维工程师一样，盯着整个训练过程别出问题；同时我们也要负责提出训练这些模型的方法配方。

这两个角色根本分不开，必须同时承担。所以我觉得，你当然可以把研究和应用看成一条连续的光谱，但在今天这个时代，不管你站在哪一侧，最后都得能在这条光谱上自由切换。

主持人：我注意到还有另一条光谱，就是软件工程师到纯人工智能研究员。你怎么看这条光谱？软件工程和人工智能研究之间，到底差别在哪？

Vlad：如果从我自己的经历出发，我会说，我们做的很多事情，以及我们提出的很多新方法，它们真正的基础其实是基础设施层面的投入。

我可以稍后更详细讲讲我团队在做什么，但先拿“蒸馏”举个例子。所谓蒸馏，本质上是一种知识迁移方式。你可以把它理解成：教师模型先在底层数据上提炼出某种统计信息，再把这些信息传给学生模型，让学生模型比完全没见过这些额外信息时表现得更好。

但如果你说的是一个超大语言模型，而且处理的是数万亿级别的词元，这背后对应的计算投入就是极其惊人的，可能是数百万、数千万美元的算力成本。

这就意味着，你必须认真思考怎么把整个系统优化到极致。因为你做的每一个操作，都会被放大到极其巨大的规模：每一秒都重要，每一个字节的存储都重要。



而这里面很大一部分工作，说到底就是非常传统的软件工程。尤其是蒸馏基础设施，到现在大概已经经历了三到四代演化。每一代里，我们都会退后一步，重新审视当前在蒸馏研究中到底用了哪些方法，再从整体上想：我们的基础设施该怎样扩展，才能支持更广泛的研究。

而且确实有几个很明确的节点：你一旦重新思考蒸馏系统的设计方式，蒸馏方法的研究速度就会明显加快。

所以这其实是一种非常典型的投入：你可能花四个月时间重写蒸馏基础设施，最后换来的，是对蒸馏缩放规律的全新认识，而这种认识又会反过来转化成非常强的模型表现。

所以它真的要求你跨越整个技术栈去工作。我几乎无法想象，如果没有这些蒸馏基础设施层面的投入，我们能做出 Flash 3.0 那样的结果。归根到底，这些东西一开始都来自一份非常老派的设计文档：你得想清楚，生成教师统计信息时，什么样的抽象才是对的；这些信息该存进什么样的存储系统；在这种规模下，跨多个数据中心读写数据时，底层该如何支撑。

这些其实都是非常经典的分布式系统问题，不是吗？

研究不是一条直线，它更像在雾里走图

主持人：对，我也是这个意思。听起来在现在这种计算规模下，确实有很多软件工程、后端基础设施类的问题。但我还是觉得，在那条光谱上的某个位置，应该会出现一个真正的跃迁——需要一些全新的能力。比如说，你把一个普通后端工程师直接丢去改模型架构，这显然和做基础设施又不是一个量级的跳跃。

你怎么看这个区别？

Vlad：对，我觉得确实有这样一个分界点。**研究这件事，本质上是一种高风险、高回报的活动。我们常会说一个词，叫“研究品味”，也就是一种高层次的直觉：在一个项目里，你面对很多不同的里程碑和可能路径时，到底应该沿着哪条路往前走。**

从某种意义上说，软件工程项目也可以被看成类似的一张有向图：你有一堆中间产物要完成，最终才能到达目标。

但软件工程里的这张图，整体上更接近确定性的。你先搭一个服务，再搭另一个服务，再搭第三个服务；先把存储层搞定，再处理上层功能。你大体可以稳定地一路往前推进。

研究就不是这样。研究里的这张图是带随机性的。因为其中一些节点——比如某个研究想法，或者到达最终目标所必需的某个环节——并不保证能成功。

我觉得这要求一种思维方式上的转变。而这种转变需要时间去学，也需要一些专门培养出来的能力。这就是很多人在博士阶段逐渐建立起来的那种东西。

如果要用一句比较简洁的话来概括，我会想到 Jacob Steinhardt 教授的一篇非常出色的文章。我很喜欢用他的框架来理解自己做的研究：研究是一种马尔可夫决策过程。

所谓马尔可夫决策过程，指的是：在一个研究项目里，不同里程碑之间存在一张带有随机性的依赖图。你可能必须先拿到某种结果，或者先证明某种结论，才能到达后面的目标。

类似地，在一个机器学习研究项目里，你可能得先把某种特征方案跑通，之后才有机会获得某种图像识别准确率，图上的节点和路径都会不断展开。



这是一种高度不确定的探索活动：某些方法可能有效，也可能无效；而一种方法一旦有效，又会为你打开一组新的可能性。

在软件工程里，你也许可以把所有通往目标的路径都列出来，然后问：最短路径是什么？

但研究里，这种方法并不最优。因为一旦图上的边不再可靠，有些节点甚至你还根本不知道——也许它们是隐藏的——那你处理问题的方式就必须改变。

具体来说，你必须同时考虑不同研究方向的成功率和时间投入，还得事先对这些概率做出估计。

而这和给一个软件项目写设计方案，是完全不同的一种思维活动。你得培养出一种直觉：在你还没真正做一件事之前，先判断它成功的可能性有多大。

很多人把这种能力和“研究品味”联系起来，我觉得这是对的。因为你如果想真正穿行于这种研究过程里，这种判断力就是必须建立起来的核心能力。

主持人：听起来你的意思是，研究里有更多的不确定性。我还是想更具体一点去理解这种工作的性质。如果你把一个后端工程师扔进一个做研究的团队里，具体会在哪些地方出现明显短板？

Vlad：我第一个想到的，就是你是否具备足够的研究背景，足够理解自己所处的研究版图。

研究工作很大程度上要求你有一种很谦逊的心态：你得承认，在你之前，这个方向已经有了大量投入；而在你真正了解这个主题上全人类目前最前沿的积累之前，你大概率不可能继续把这个前沿往前推。

所以，在某个具体方向上建立扎实的已有工作认识，并真正做好相关文献梳理，可能就是很多人最先会卡住的地方。你得有能力沿着一个主题的引用脉络一路追下去，知道该读什么、怎么读。

因为你根本没时间把所有论文都读完，所以你必须培养一种判断力：哪些论文最值得读？在不阅读全文的情况下，怎么判断一篇论文到底值不值得投入时间？

这是我最先想到的一项必须建立起来的能力。

而且，光是要看懂这些研究级论文，你本身就具备机器学习和计算机科学背景。再进一步，还要看论文属于哪个领域、讲什么内容；有些论文背后需要的数学基础和课程训练都很重，没有这些前置知识，你根本不可能真正理解它的方法。

这件事非常重要，因为如果你不了解现有的方法论是什么，你几乎不可能在它的基础上做出改进。

比如我前面提到，我们团队做蒸馏。如果你想推进大语言模型蒸馏方向上的理解，你首先得很好地理解我们到底想用大语言模型去做什么。

我简单概括一下：**大语言模型研究，尤其在预训练阶段，核心就是缩放规律。**

什么叫缩放规律？很多人会特别关注它是不是幂律、指数是多少，但真正重要的其实不是函数形式本身。真正重要的是：对于一套扩大语言模型的方法，当你不断往一次预训练里投入更多



计算量时，你必须能够预测，这个语言模型最终的测试损失会落在什么位置。

为什么这个问题重要？为什么我们要预测模型的泛化误差？

在传统机器学习世界里，比如说我们做图像识别，打 ImageNet，你的测试损失就是那一千个类别上的分类错误率。你提出一个网络结构，比如 VGG 或 ResNet，训练一下，再看它的分类错误率是多少，这就能估计这个模型在这类任务上的表现。

我们还可以通过验证集来估计方法好不好。每当出现一个新的网络结构想法，我们训练一下，做一系列验证实验，得到交叉验证误差，而这个误差本身就是对最终测试误差的估计。于是你可以通过这个流程不断迭代不同想法。

但大语言模型不是这样。因为每一次真正做预训练，你投入的计算量都会比之前任何一次都更大。

所以从某种意义上说，它像是一个“一次性”的 ImageNet 问题。你永远没有机会先完整地看一遍真正的 ImageNet，再慢慢调。你只能先在 MNIST 上练，再到 CIFAR 上练，然后试图根据这些经验，设计一种办法，希望它第一次上 ImageNet 就能直接奏效。

如果你真这么做——我相信很多人都试过，我自己当年学这些东西的时候也一样——你会发现，它在 MNIST 上很好，在 CIFAR 上可能也不错，但一到 ImageNet 就突然崩掉。

你会慢慢意识到，很多东西并不会随着规模自然泛化。

所以，我们在大语言模型里做的大量工作，其实是在设计“配方”。**所谓配方，就是一个函数：输入是你愿意投入多少计算量，输出一整套语言模型训练流程。**

如果你能把这套配方和一种预测规则结合起来，而且这个预测规则能够准确预测最终效果，那你就可以据此做决策，去改进你的训练配方。

我刚刚这一大段，其实是在给大语言模型研究的基本面貌提供背景。但这种理解之所以存在，是建立在大量早期语言模型缩放研究之上的，比如 Kaplan 的论文，比如 Chinchilla。

自从那两篇论文之后，又有很多工作开始研究：除了参数量和训练词元数，究竟还有什么因素会影响预测准确性，比如不同词元的独特数量等等。

但我会说，这两篇论文对大语言模型来说都是基础性文献；而它们本身又建立在更长的一条缩放研究脉络之上，可以一直追溯到最早期的 GPT。Google 这边也通过 PaLM 系列论文积累了大量这方面的研究。

这些工作共同塑造了我前面描述的那种视角。而这种视角，基本上只有你自己亲自沿着那条文献线索走过一遍，才会真正建立起来。

主持人：假设你要给自己的团队选人，而你判断一个人是否适合帮助你推进前沿的标准，是他对前沿本身的理解，包括对已有文献的掌握，而这又要求很多前置能力。我记得你在文章里把其中一部分叫作“数学成熟度”。

Vlad：对。我觉得，一旦你具备了数学成熟度，理解这些论文、读懂这些论文，其实并不难。



我刚才提到的那些论文，现在基本都已经算是入门门槛了，所以如果是候选人，我默认他们应该熟悉这些内容。

但更一般地说，关键能力在于：**你能不能钻进这种级别的论文里，把它真正读懂；你能不能把论文里的研究想法自己实现出来。这是一项非常重要的能力。**

我们会看到各种各样的想法，它们未必都能直接应用到我们的领域。但如果你能深入理解它们，你就可以在这些想法上继续迭代，并把它们改造到我们的应用场景里。

所以当我们评估一个人是否能处理这些机器学习论文中的数学概念时，这大概就是最关键的能力信号。它说明你能拿起一篇陌生论文，判断其中哪些想法可以迁移到 Google 的实际环境里。

真正值钱的能力，常常在系统最底层

主持人：这肯定不是一份穷尽清单，但我还是很好奇：还有哪些领域，是人们可以去深入钻研、而且对前沿人工智能研究确实重要的？你提到了蒸馏，也提到了内核。听起来内核几乎在哪都很有用，但除了这些之外，还有没有什么你会顺手列出来的方向？

Vlad：我觉得一个非常有力量的方向，其实是编程语言研究。

因为如果我们能在编程语言层面创造出更好的抽象，就能极大促进内核开发。我觉得 ThunderKittens 就是一个很好的例子。它提供了一种抽象方式，让你写内核的时候，只需要围绕四个函数工作，而不是去面对一大团随意拼起来的 C 代码。这样一来，你在开发那些能够充分利用硬件的算法时，速度就会快得多。

这时候重点已经不只是编程语言研究本身，而是你是否对这种抽象充满兴趣，并愿意和那些关注底层硬件的人一起工作，比如去尝试 CuTe 这类领域专用语言。

这类方向里有很多都是围绕特定硬件设计的专用语言。

除了编程语言和缩放规律相关文献之外，我还会想到强化学习文献。

特别是自从“基于人类反馈的强化学习”出现之后，我们已经看到像 PPO 这样的深度强化学习算法，确实可以进入生产系统。曾经有一段时间，这件事到底成不成立，其实还是有争议的；但现在几乎已经形成共识：这些算法确实会被应用在真实的生产环境里。

而想理解它背后的理论，你通常得从强化学习的基础一路往上学，再走到今天非常丰富的各种价值型方法和策略梯度方法。

这是另一个我觉得文献脉络极其丰富、非常值得慢慢爬进去的方向。

再往“后端工程师”一点的方向说，除了内核本身之外，我觉得分布式系统和优化之间，还有一个非常有意思的交叉地带。比如：如何设计神经网络训练算法，让它能在大量图形处理器上训练——这里面会有很多有趣的问题：异步性、梯度的新鲜度、流水线方式会怎样加剧梯度陈旧，等等。

你在训练算法设计里做出的这些系统性选择，都会影响神经网络的收敛情况和最终质量。

这些问题其实就算脱离大语言模型场景，也可以单独研究，而且已经被研究很久了。所以如果你更偏基础设施方向，那我会说，先把这些算法是怎么运作的弄明白，会是一个非常好的起



点。

主持人：你觉得不同前沿实验室之间，对人的要求会有差别吗？比如说，如果有人想去 Google DeepMind，是不是会有某个方向比 Anthropic 更受重视？至少从技能组合上看，我感觉应该还是挺接近的。

Vlad：对，我觉得不同实验室可能会在商业策略上有所差别，提供的产品和服务也会因为各自的专长和客户类型不同而不一样。

但如果问大家真正看重什么，我认为实验室之间重叠其实非常大。我那篇文章发出去以后，你会看到 OpenAI 和 Anthropic 的人也在说：对，这些建议我们也认同。

所以我觉得，这至少算一个小小的证据，说明大家在这件事上其实非常接近。

主持人：我觉得很多人之所以特别想往人工智能研究靠近，是因为他们在想：未来软件工程也许没那么重要了。那研究这边会不会也有类似问题——大语言模型以后会不会也把很多研究工作接过去？所以人工智能研究未必就比软件工程更值得押注。

Vlad：我觉得研究能力只会越来越重要。

能够处理工作规划里那些不确定、随机的成分，会越来越成为我们工作的核心部分。你要学会怎么在你做的任何事情里利用人工智能——而且这件事甚至不必局限于软件——这种能力应该立刻开始练，因为这些系统本来就不是确定性的。

你真正要思考的是：我怎样围绕这些大语言模型搭系统，让自己把工作做得更有效？未来真正能把你和别人区分开的，就是这种能力。我觉得不管你具体做什么，这一点都成立。

坦白说，现在到处都是恐惧营销。尤其有些人在谈人工智能的时候，本来就在故意制造这种恐慌。我的感觉是，人们真正应该做的，是把注意力放回自己身上，想办法让自己变得更高效。

我并不认为人工智能会取代我们所有人的工作。原因在于，人类在组织里承担的一个关键角色，是构成一张信任网络。组织本身就是一组资源和一群人，而这群人负责管理这些资源。

我们的一项重要职责，就是把这些资源分配到特定目标上。

就算人工智能能大幅加快执行速度，关于资源如何分配的决定，最终还是必须落到具体的人身上。这件事永远要有人负责。因为你不能把责任推给人工智能。

比如说，现在的大语言模型已经非常懂法律了，它可以帮你审合同之类的；但它不能代表你出庭，因为它不可能被吊销律师执照。

我觉得这就是一个非常尖锐的例子，说明为什么法律职业依然存在：即便大语言模型很擅长调用先例、理解法理，**你还是需要一个能为结果负责的人，去验证人工智能的输出，用人工智能来更高效地完成法律工作，而不是把自己的法律辩护整个交给一个语言模型。**

别被恐惧营销带着走

主持人：对，我觉得当初促使你写那篇文章的原因，其实也正是这种恐惧气氛。

Vlad：对。我真的觉得，人们应该拥有的是一种更建设性的心态。



我之前看到一条推文，好像是 Deedy 发的，里面是那种很长的恐慌叙事，说什么人工智能会制造一个永久性的底层阶级之类的。人很容易被卷进去。

但我觉得真正该想的是：**我们每个人都对自己的未来拥有主动权。今天就可以开始投资那些对明天有意义的能力。这才是唯一真正该做的事。光担心，什么也改变不了。**

所以我想写那篇文章，也是为了回应这一点。因为我确实感觉到，这种情绪一直在扩散。前阵子我去普林斯顿做讲座，现场一个很大的问题就是：我怎样才能去 Google DeepMind 工作？

而且这几乎就是大家知道我在做什么之后，最常问我的问题。

所以我想，也许给出一些更具建设性的方向，会对整个讨论更有帮助。

主持人：关于那篇文章最后还有一个问题。因为如果你想拿到一个岗位，当然一方面是能力本身，我们前面聊了很多你的能力是否匹配这个岗位；但另一方面，其实还有“信号”——你如何向外界展示自己，以及什么样的信号是被看重的。

如果你要把自己推给这些前沿实验室，什么样的信号最重要？

Vlad：**最重要的是，拿出真实证据，证明你沿着这些方向做出过对别人有用的东西。比如内核开发。**

现在已经有 many 开源大语言模型了，你完全可以拿其中任何一个来做优化。你甚至不一定要把模型本身做得更强。很多场景下，只要你能证明：在某个设定里，我把这个地方做快了，把那个地方做顺了，这就已经很有价值了。

而且事情也不只是在图形处理器上加速模型推理。围绕大语言模型的服务栈，本身就是一个非常复杂的分布式系统。它要维护键值缓存，还要处理各种负载均衡、请求排队——这些都是后端服务里再常见不过的问题。

这些项目一直都需要帮助。所以像给 vLLM、SGLang 这种项目做贡献，或者基于 TensorRT 做一些实践展示，都会很有价值。TensorRT 那边我记得还有个分布式系统叫 Dynamo，它支持把推理服务拆分部署。你如果能展示自己基于这些组件做了项目，或者真正改进了这些组件——

对我来说，这会是非常强的候选人信号；同时我也觉得，这会对开源社区非常受欢迎的贡献。

主持人：还有一点，我们前面很多讨论都默认了一条路径：从外部跳进前沿实验室。

但其实这些实验室背后往往都有很大的组织体系，很多团队并不是直接做最前沿研究的。比如在 Google 体系里，有些基础设施工程师可能本来在搜索团队工作，有很强的后端能力，但领域背景没那么深，然后他们想内部转到 Google DeepMind。

在这种内部转岗的情况下，你的建议会和外部候选人不一样吗？

Vlad：我以前在搜索那边有一位合作很紧密的同事，后来真的转到了我团队，叫 Nate Lintz。他非常厉害，现在在我们团队里负责了很多核心工作，特别是在 Flash 和 Flash-Lite 的推理协同设计方面。

我觉得他就是一个很好的例子。他当时的思路是：我怎样才能帮助自己所在的产品线，尽可能有效地采用这项技术？

所以我会说，如果你所在的组织不是直接造模型的，而是试图利用这些模型，那这里面其实有一个很大的空白：**如何真正把这些大语言模型高效地应用到你的组织里，如何高效地把它们服务起来。**

如果你能成为那个把这件事做得特别好的人，这不仅会给你的团队创造巨大的业务价值——这显然会让你在本组织里脱颖而出——同时，你也会自然而然成为我们研究团队在对接产品落地时最重要的合作方之一。因为我们会需要和你这样的人协作，确保模型能在你们组织里真正发挥作用。

所以到了那一步，你也许会想转岗，也许不会。你如果想转，我们当然很欢迎。

但即便你不转，**其实你已经在做一件非常前沿的事情了：把这种新技术整合进真实产品，让真实用户去使用。**

所以，对这种情况，我的建议大体就是这样。

先做出东西来，信号自然会出现

主持人：在那篇文章快结束的时候，作为收尾，你还发出了一个很具体的邀请，因为我知道你当时在招人。要不要把那段再讲一下？

Vlad：对，我当时是在想：我怎么才能真正做到“说到做到”？怎么证明我不是只在嘴上说这些能力重要，而是真的愿意按这个标准去看人？

我想表达的是：**如果你能通过一些具体练习，证明自己至少在我强调的那些能力上已经有了一些初步证据——比如意图、数学成熟度、韧性——那我愿意认真看。**

所以我列了几项练习：有些是为了展示你对缩放规律已经有初步理解；有些是要你在工程层面愿意深入细节，真正亲手实现一个 Transformer；还有一些则是展示你愿意掌握我们每天都会用到的那些基本数学，用来估算和设计这些大语言模型。

我现在已经记不清完整的练习清单了，但如果你把《缩放之书》里的题目认真手写做完，录一段自己讲解过程的视频，再加上我文章里提到的 Transformer 练习，把这些都发给我——

那就是非常有力的东西。如果你能在纽约办公室工作，我会非常愿意面试你。

确实已经有不少人因此联系我了。我其实已经收到过几份提交，现在也已经在推进其中一些人的面试流程。

所以，这件事工作量当然不小。但让我很惊讶的是，我大概在文章发出后一周内，就收到了回应。说明这件事确实是做得到的。

当然，我的招聘名额不是无限的，所以这个邀请虽然有效，但我也可能招无限多人。

不过好的一点是：**这本身就是一种非常强的自我成长信号。所以，不管你最后会不会因此去 Google DeepMind，这些事情都值得你为了自己去做。**

而且我觉得，它也会帮助你准备去其他地方面试。

当然，如果你真的完成了这些练习之后联系我，即便我这边已经招满了，我也认识很多其他正在招人的人，我也很乐意帮忙推荐。

真正有意思的，是把模型做对，也把硬件吃满

主持人：接下来换个话题。我看到你是 Gemini 预训练方向的负责人，我觉得挺有意思的，很想听你从自己的角度高层次讲讲：在你看来，预训练到底是什么？以及这个方向上最重要的挑战有哪些？

Vlad：可以。

作为预训练方向的负责人，我们要做的事情其实很多。

我团队负责交付的具体产品，包括 Flash 模型和 Flash-Lite 模型。这些模型会被用在搜索里的 AI 概览 and AI 模式上，也包括一些供其他组织内部使用的一方模型，比如广告 and YouTube。

除此之外，我们还是 Google and Apple 合作中的关键技术负责人之一，所以也会做那边的技术工作。

这些是我团队承担的产品层面交付。

再往上，我们还要做研究，确保这些交付本身处在业界最先进水平。同时，我们也做一些更通用的预训练研究，再反过来贡献给 Pro 系列模型。

如果从研究性质来分，我会说大体可以分成三个方向。

第一是前面提到过的蒸馏。

第二是我喜欢称之为“推理协同设计”的东西。也就是：设计那些在推理时足够高效的神经网络架构。具体来说，就是决定网络的拓扑结构，决定 Transformer 里门控层和线性层的矩阵形状，决定注意力机制的形状、注意力头数量，诸如此类。目标是让这些设计在你实际部署的硬件上，能尽可能高效地运行。

第三个支柱，是新的量化方法。量化一直是我特别有感情、也从加入 Google 以来一直在研究的方向。它实际上会改变前两个方向能够做到什么，因此推进模型压缩的最前沿，也就成了我们团队研究中的重要组成部分。

一般来说，量化指的是：在某种意义上减少神经网络权重表示所占据的体积。

通常在训练时，神经网络里的那些矩阵参数是用三十二位浮点数来存储的。

但事实证明，**在实际计算中，你根本不需要那么高的精度，模型质量依然可以保持得很好。**用一些并不算复杂的方法，就能把这些权重的存储精度压缩到四位。

也就是说，本来三十二位浮点数能表示一大段数值范围、达到大约七位精度的东西，现在居然能以相当高的保真度，用四位整数来表示——而四位整数覆盖的范围其实非常小，大约只是从负八到七。这本身已经像奇迹一样了。



但更神奇的是，你甚至还可以把类似的量化变换应用到神经网络运行时的激活值上。

一旦做到这一点，实际做矩阵乘法时，参与运算的数就会比以前小得多，因此运行神经网络所需的电力也会显著下降。

有意思的是，人工智能硬件总运营成本里，有百分之九十九都来自驱动这些芯片所消耗的电力。如果你能把这些运算做得更便宜，神经网络就能更廉价、更高效地运行。

这会直接帮助你处理更多请求，也会改善时延。

所以量化研究的核心问题就是：我们怎样把前沿继续往四位之外推进？

主持人：我在推特上经常看到一种说法，老在讨论“模型浮点运算利用率”。对于不在这个行业里的人，或者说不了解这个概念的人来说，他们一看到那个数字只有十几个百分点，就会觉得：哇，这不是浪费了绝大多数图形处理器资源吗？

我想请你帮大家解释一下，为什么看起来“低”的模型浮点运算利用率，其实一点也不低。顺便也请你解释一下它到底是什么。

Vlad：对。所谓模型浮点运算利用率，本质上是这样算的：把神经网络实际完成的浮点运算次数，除以在这段请求时间里，硬件加速器理论上本可以完成的总浮点运算次数。

所以从某种意义上讲，这个指标告诉我们：在多大比例的时间里，我们真正有效利用了这块加速器的浮点运算能力。

如果你想达到百分之百的利用率，那意味着你必须把硬件里的矩阵乘法单元一直吃满。

也就是说，它只能不停地在循环里做矩阵乘法，不能去读内存，也不能做任何别的操作。

但这种计算在现实里没什么意义。神经网络需要做激活函数，需要做注意力计算，需要把中间结果写回高带宽显存。所有这些操作，都需要使用内存总线，或者向量处理单元。甚至有些数学操作，本来就是底层硬件里比矩阵乘法慢得多的部分。

这些因素都会导致模型不可能一直按处理器标称峰值运行。

所以你之所以看不到百分之百的利用率，是因为神经网络有一部分时间在读写内存，另一部分时间在做一些天生就比矩阵乘法慢的操作。

我前面提到的推理协同设计，其实很大一部分就是在协调芯片的各种能力：和其他芯片之间的通信、内存带宽、从内存中读取参数的速度、浮点运算能力——这里既包括矩阵乘法，也包括向量运算，比如做激活函数。

这些能力在硬件上都有不同的速率，而任意一种计算都不可能天然精确匹配硬件每一类能力的最佳节奏。

在设计神经网络时，你要做的是选出一组结构形状，让这个网络能尽可能同时吃满这些硬件单元，从而在推理时把利用率尽量拉高。

而让这件事不只是一个代数题的原因在于：这些设计选择最终会影响你训练出来的神经网络质量。



所以推理协同设计真正有意思的地方就在于：我们怎样设计出一种神经网络架构，它既能稳定地随规模扩大，又有好的质量预测，同时还能让推理时的硬件利用率尽可能高。

这种联合优化，正是推理协同设计最有趣的地方。

而且它还是个常青问题。因为随着硬件变化，浮点运算、内存带宽、通信带宽之间的相对关系也会变化，而这又会改变“什么样的神经网络形状才是最优”的答案。

很多关键工作，并不光鲜

主持人：换个话题。Google 有一种叫“即时奖金”的机制，就是有人可以因为你做得特别好，给你一笔一次性的奖金。我在你的履历里看到，Jeff Dean 这位传奇人物曾经给你过即时奖金。

如果你愿意讲讲那个故事的话，我很想听。为什么他会给你这个奖金？

Vlad：那其实发生在 Gemini 项目刚开始的时候。

Jeff 当时给了一批参与 Bard 第一版发布的人即时奖金。我在那个非常庞大的项目里，其实只做了一个很小的贡献。我帮忙做了监督微调，参与了 Bard 最早几个发布版本中的某一版。

那段经历给我最大的启发是：在那个时候，我还只是 Google Brain 里一个纯做研究的人，我当时非常执着于一件事——怎么尽量多发第一作者论文到 NeurIPS、ICML、ICLR。

我非常清楚地记得，当时我脑子里的第一反应其实是：我是不是应该继续低头写论文？

很幸运的是，那时我的经理 Rohan Anil 非常鼓励我们都投入这个方向。对我来说，这正好给了我需要的推动力——让我真的卷起袖子，去做很多超参数调优 and 工程工作，想办法让这个模型在一些非常老的张量处理器上跑起来，好为监督微调多争取一些训练机会。

那次很小的初步参与，后来得到了 Jeff Dean 的认可。我觉得它也进一步推动了我在大语言模型方向上的投入，最终把我带到了今天的位置。

所以我会说，这件事的重点并不在于，我当时那一点监督微调工作对最初发布到底帮了多大忙。

更重要的是，它让微观的我认识到：想参与那些高价值项目，你往往需要投入很多并不光鲜的工作。可能只是超参数调优，只是为了让程序塞进特定内存预算里，不停去挤压编译器表现、抠那点空间。

但这些工作，都会服务于更大的业务目标。而这件事，对于进入真正重要的项目来说，非常关键。

主持人：你在 Gemini 上也做了很久了，而且这还是一个最高优先级项目，所以你一定经历过一些事故或者“战争故事”。我很好奇，你最喜欢的一段 Gemini 故事是什么？

Vlad：如果让我选，我最喜欢的应该是 Flash 2.0。

那真的是一段非常艰难、也非常漫长的旅程。但我们当时最核心的优化目标之一，是延续 Flash 1.5 已经建立起来的那个定位：做一种非常快、时延很低、但质量依然很强的模型。

尤其是它必须快，因为搜索需要它在 AI 模式里非常迅速地返回回答。

正因为如此，在 Flash 1 的时候，虽然我们已经知道混合专家模型，也知道它可以显著提升容量，但当时一个很现实的问题是：我们当然想用这种新架构，可你很难直接切过去。

因为混合专家模型的特点之一就是参数通常更多。而参数一多，就会占用更多高带宽显存。可我们部署所用的这些芯片，高带宽显存是有限的。

所以你就必须把这个混合专家模型切分到多块芯片上。假设你有 N 个专家模块，那你可能得把它们分布到 N 块芯片上，或者按某个和 N 相关的比例分布。

这样一来，模型中间就会出现大量通信。当一个词元需要被路由到某个专家模块，而这个词元原本在第一块张量处理器上，却必须送到最后一块去处理——你就在前向传播过程中引入了巨大的通信量。

而这个操作的时延会随着 N 的增加而急剧变大。**混合专家模型的难点就在于，它会把 N 推得很高。这就成了一个非常严重的瓶颈。**

有意思的是，我们其实很早就知道流水线式服务这个思路。只是对稠密模型来说，它一直没有真正重要到必须采用。

我很清楚地记得，自己很早曾和 Sholto 聊过这个事。Sholto 当时说：对啊，你这里主要受限于浮点运算量，所以做流水线并不会改变输入预填阶段的性能曲线。

后来证明他是对的。我试了一下，然后就把这个想法搁置了。

但有意思的是，当时我团队很小，我手下的一位同事 Geng Yan 提出了一个非常好的想法。他当时和 Rahul Arya 以及 Google 以色列团队的几位同事合作，**提出把流水线式预填应用到混合专家模型上。**

所谓流水线，就是说不再把那 N 台机器并行地分给同一层里的 N 个专家模块，而是把不同的层分配到不同机器上。

也就是说，不再是在某一层里把词元在各台机器之间来回路由；而是某一层的机器先处理你预填请求中的一部分词元，再把这些处理过的词元传给下一台机器，让它处理第二层，再传给第三层、第四层。

这样一来，所有专家模块都可以常驻在单台机器上，或者更少数几台机器上。它本质上改变了通信模式：从原来那种每一层都要做大量词元交换，变成了一种可以被其他计算掩盖掉的通信方式。

因为你可以把流水线式预填分摊到请求的不同部分。比如第二层在处理请求前一千个词元时，第一层所在的第一台芯片已经可以开始处理第二个一千词元了。

所以，这其实是一种打破高带宽显存约束的方法：不是把专家模块在机器之间搬来搬去，而是把网络层分布到不同机器上。

正因为如此，**通信开销降下来了，混合专家模型的时延突然就变得非常有吸引力。**



Gemini 2.0 的技术报告里写得很清楚：它是一整个混合专家系列模型。而使这件事成为可能的原因之一，就是这种推理阶段的服务创新。

Dwarkesh and Reiner 有一篇非常棒的文章，专门讲这个优化，而且你甚至可以把它写进《缩放之书》的代数框架里去理解。

这也是一个很好的例子，说明这种看似局部的改动，实际上会对大语言模型质量产生多么巨大的影响。

真正让 Flash 2.0 那段经历如此有成就感的，是这个重大的混合专家决策。它在当时听起来像个小技术选择，但大家真的非常担心：这个模型的时延到底能不能压到合理范围。

好在我最后推动了一套非常透明的技术决策流程，把问题彻底查明了。最后我们做出了正确的判断。

可接下来还得训练它。

这是一个在 Flash 这个规模上，我们训练过的最大模型。我们知道这是对的，但接下来的四十天会是极其艰苦的过程，而且团队非常小。

负责这次训练轮班的人，可能只有五个左右。我记得我们几乎是一天接一天地交接，轮流做这种运维式工作：想办法让训练任务一直活着。因为在当时，这真的是一个非常需要人工盯着的过程——你得确保所有东西都稳定推进，确保数据迭代器调好了，不会拖慢任务；如果数据里有空洞、索引有问题，你必须立刻修补，因为每一分钟都在烧掉巨量算力。

主持人：那夜里和周末怎么办？

Vlad：对，所以那四十天里，我们基本没怎么睡。

我们必须在巴黎办公室 and 山景城之间做双班倒。

而让这一切最终变得特别值得的，是模型发布的时候，差不多正好赶上 DeepSeek-V3 出来。那时《华尔街日报》发了一篇很夸张的“红色恐慌”式文章，讲什么中国要靠开源模型接管人工智能之类的。

我记得我朋友给我发来一张截图，是大模型竞技场排行榜的一张表。最右上角是 ChatGPT，DeepSeek 紧随其后。然后文章还在说，DeepSeek 只花了几百万美元训练，就已经追得那么紧。

我朋友就跟我说：哦，Gemini 落后太多了。因为那张表最底下放的是一个我记得像 Flash 1.5 Pro 之类的版本。

然后我一看，心想：这挺有意思的。我刚好一直在看这个排行榜，因为我们刚发了一个模型，而网站上的实际情况显然不是那样。

后来才发现，《华尔街日报》那篇文章里把一些行给省略掉了。所以如果你今天再回去看那篇文章，你就会看到，当时真正处在业界最前沿的模型——Flash 2.0 Thinking——其实在表格的右上角，远远领先于 DeepSeek-V3。这多少会让他们当时想讲的那个“开源压倒一切”的叙事不那么成立。



但对我们团队来说，那确实是一次非常重要的成就。

做那个别人真心希望他成功的人

主持人：最后一个问题。如果你可以回到自己刚大学毕业的时候，大概就是本科毕业那会儿，然后给那个时候的自己一些建议——以现在的你所知道的这些东西——你会说什么？

Vlad：你得去追那些这个世界今天真正面对的问题。去追那些人们在日常生活中真正遇到的挑战。不要害怕自己切进去的只是这个问题里较小的一部分，或者听起来没那么体面的那一部分。

即便它不是那种“很高级”的研究、数学之类的东西，也没关系。你要相信，只要你是做重要的事，哪怕只是一个更大项目里的较小组成部分，你最终也会看到，真正推动前沿往前走的，到底是什么。

我想，这里面需要一种面对问题时的谦逊。你真正该追求的是这个。

如果再说另一条建议，也许更偏职业层面一点，那就是：**成为那种别人愿意看到他成功的同事。**

我的意思是，大家总会谈什么“职场精神病”、马基雅维利式领导者，或者那种为了结果不惜一切代价的人。他们也许能通过压榨别人，换来一些短期收益。

但这么多年和各种样的人共事下来，我觉得最有意思的是：我见过极少数那样的人——其中有一位特别亲近，是我的朋友 and 导师 Todd Lipkin，也是最早带我进入计算机科学的人——他们非常善良，而且你能从他们身上学到很多。你会真心觉得：我愿意跟着这个人一起做事，也愿意帮助他成功。

特别是，**如果你是那种能帮助别人把项目做成的人，能提出一些项目，让别人可以用自己的互补能力在其中发光的人，大家会注意到这一点。**将来他们也会更愿意参与到你提出的项目里，也会更愿意支持你。

很多人一想到职场互动，就会变得很犬儒，总想着博弈，想着怎么占优。

但我的经验是，这种更友好、更合作的方式，往往会培养出一种很深的协作感 and 互相帮助的意愿。而要把那些需要多人、多种能力共同跨线的大项目真正做成，这种东西太重要了。

所以，如果我能给更早版本的自己一点人际层面、职业层面的建议，那就是：**去做那种人。做那个别人会真心希望你成功的人。**

主持人：我很喜欢这番建议，因为它正好对抗了那种很犬儒的建议。我也很喜欢，你最初那篇文章本身，就是在对抗那种“末日论”“永久底层阶级”式的说法。非常感谢你今天抽时间来聊。真的很有意思，也非常感谢你。

Vlad：谢谢你邀请我，Ryan。

本文来源：[CSDN](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。



写评论

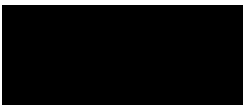
请发表您的评论

 表情

 图片

发表评论

1 条评论



正在加载